

RETAIL SALES ANALYTICS

Using Microsoft Fabric

End-to-End Data Engineering, Lakehouse Transformation and Power BI Reporting

Project	Retail Sales Analytics
Platform	Microsoft Fabric
Core services	Azure Storage • Data Factory • Lakehouse • PySpark • Power BI
Architecture	Bronze → Silver → Gold
Final output	Retail Sales Performance Dashboard
Report date	1 October 2026

Evidence-based documentation

This report is organized from the supplied project screenshots into the actual implementation sequence. Descriptions distinguish visible evidence from interpretation. Credential screens and personal account details are intentionally excluded from the public evidence section.

1. Executive Summary

This project demonstrates an end-to-end retail analytics workflow using Microsoft Fabric. Retail source files are stored in Azure Storage, ingested through a Fabric Data Factory pipeline, landed in a Fabric Lakehouse, transformed with PySpark, organized into Silver and Gold datasets, exposed through a Power BI semantic model, and finally presented through a Retail Sales Performance Dashboard.

Project objectives

- Integrate orders, inventory and returns data from different source formats.
- Create a controlled Bronze landing layer in the Fabric Lakehouse.
- Clean and standardize inconsistent fields using PySpark.
- Join orders with return summaries and prepare product-level analytical outputs.
- Expose Gold metrics through a Power BI semantic model.
- Build a clear dashboard for product, stock, returns and cost analysis.

Observed implementation objects

Object	Observed name
Fabric workspace	Retail Project
Lakehouse	Retail_LH
Pipeline	Retail_Pipeline
Notebook	Retail_NB
Gold table	gold_product_kpis
Final report	Retail Sales Performance Dashboard

2. Solution Architecture

The screenshots support a layered architecture in which Azure Storage provides the source files, Fabric Data Factory performs ingestion, the Retail_LH Lakehouse provides Bronze/Silver/Gold storage, Retail_NB performs PySpark transformation, and Power BI provides the semantic and reporting layer.

Layer / Stage	Project Object	Purpose
Source	Azure Storage / source container	Stores orders, inventory and returns files
Ingestion	Retail_Pipeline	Moves source data into Fabric
Bronze	Retail_LH / Files / Bronze	Raw Parquet landing area
Silver	silver_orders / silver_returns / silver_inventory	Cleaned and typed datasets
Gold	gold_product_kpis	Curated product-level analytical output
Semantic Model	Power BI semantic model	Exposes analytical fields and calculations
Reporting	Retail Sales Performance Dashboard	Interactive business reporting

End-to-end flow

Azure Storage → Retail_Pipeline → Retail_LH Bronze → Retail_NB / PySpark → Silver tables → gold_product_kpis → Power BI semantic model → Retail Sales Performance Dashboard

1. Azure Storage Account and Data Lake Endpoint

The project begins with an Azure Storage account used as the source environment. The endpoint configuration shows the Data Lake Storage endpoint that is used by the Fabric pipeline connection. For the public report, credential values are intentionally not reproduced.

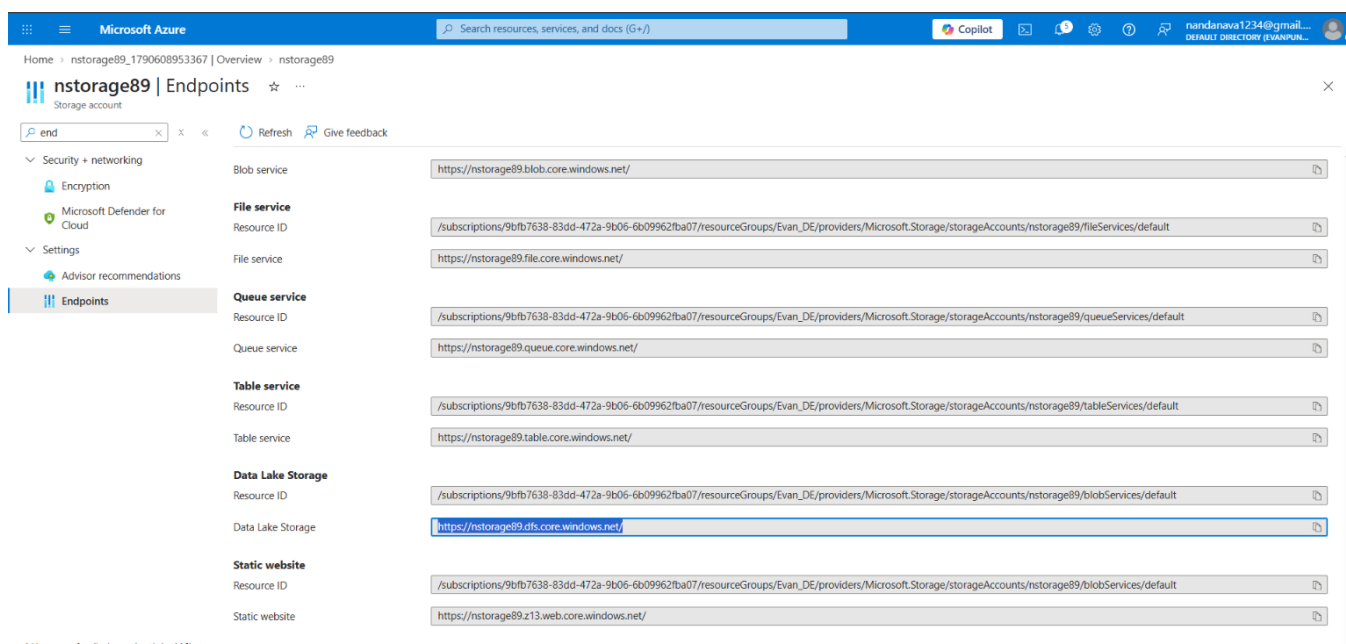


Figure 1. Azure Storage account endpoints.

Project significance: This step connects the implementation stage to the next stage in the data flow, so the report can be followed from source ingestion through transformation and reporting.

2. Source Data Container

The source container contains three retail datasets: `inventory_data.json`, `orders_data.csv` and `returns_data.xlsx`. These datasets represent inventory, sales/order transactions and product returns.

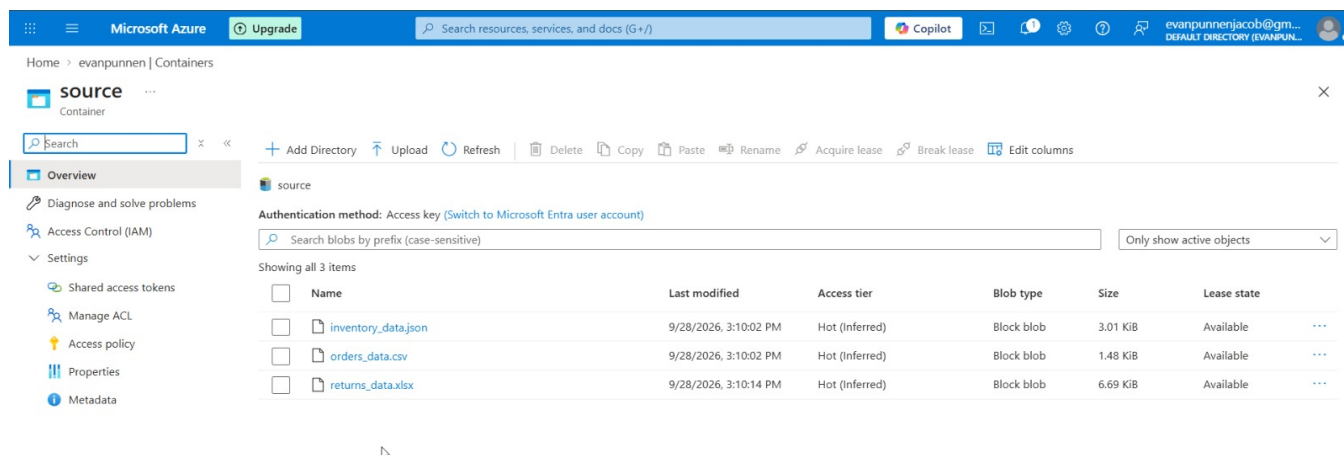


Figure 2. Source container containing the three input files.

Project significance: This step connects the implementation stage to the next stage in the data flow, so the report can be followed from source ingestion through transformation and reporting.

3. Create the Fabric Data Pipeline

A Fabric Data Factory pipeline named Retail_Pipeline is created inside the Retail Project workspace. The pipeline is the orchestration layer responsible for moving the three source datasets into the Fabric Lakehouse.

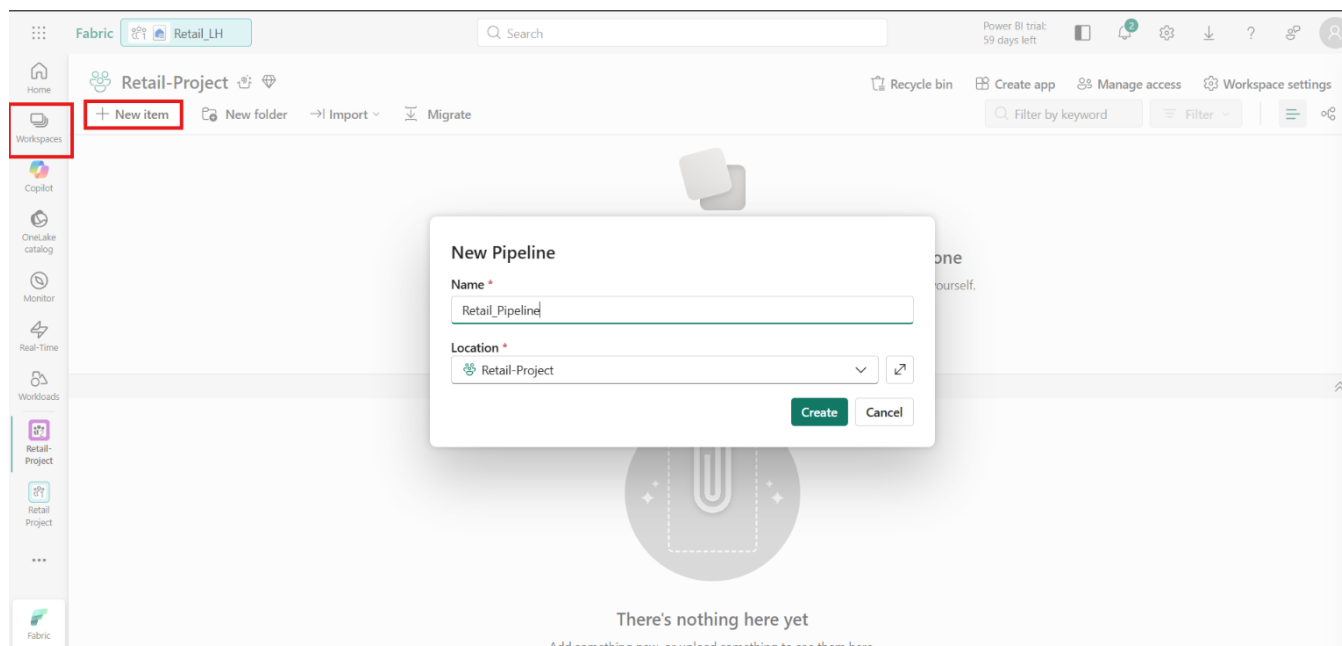


Figure 3. Retail_Pipeline creation.

Project significance: This step connects the implementation stage to the next stage in the data flow, so the report can be followed from source ingestion through transformation and reporting.

4. Configure the Orders Source

The Orders Copy Data activity is configured to read orders_data.csv from the source location through the Azure storage connection. The screenshot confirms a successful connection and a delimited-text source.

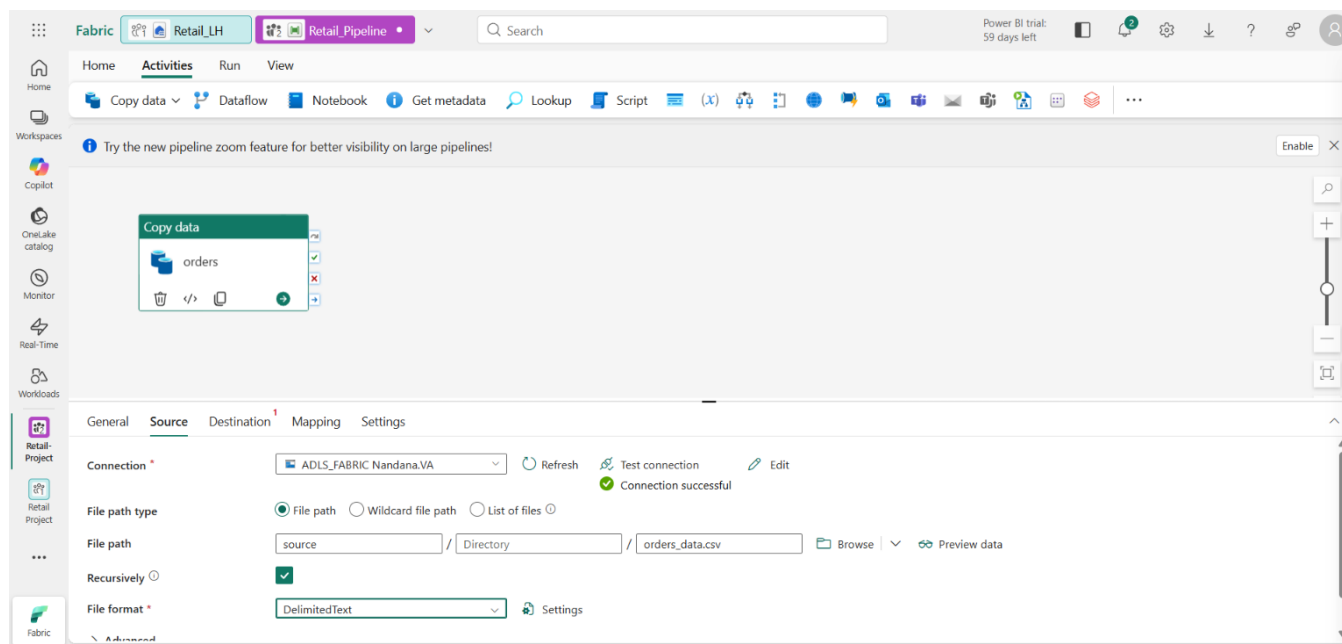


Figure 4. Orders source configuration.

Project significance: This step connects the implementation stage to the next stage in the data flow, so the report can be followed from source ingestion through transformation and reporting.

5. Configure the Bronze Destination

The Orders activity is configured to write into the Retail_LH Lakehouse, under the Bronze file path, using Parquet as the destination format. The same Bronze landing pattern is used for the other datasets.

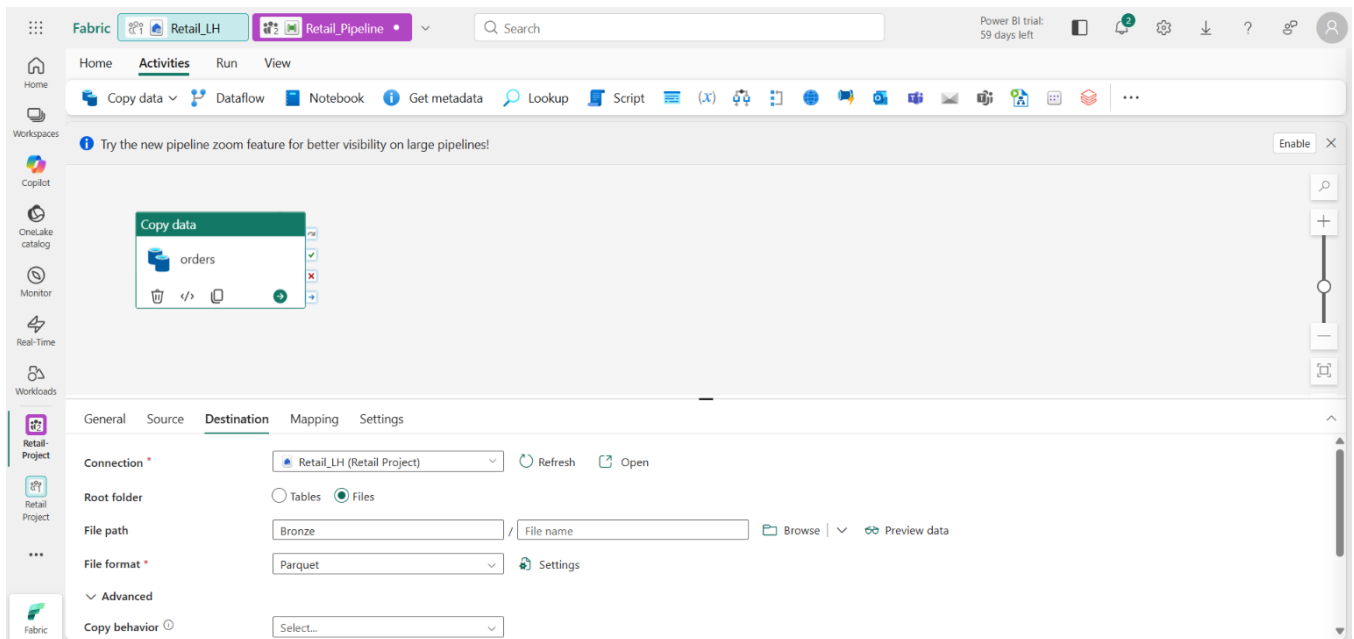


Figure 5. Orders destination configuration.

Project significance: This step connects the implementation stage to the next stage in the data flow, so the report can be followed from source ingestion through transformation and reporting.

6. Build and Run the Ingestion Pipeline

The completed Retail_Pipeline contains three Copy Data activities: orders, Inventory and returns. The activities are connected in sequence, and the run output shows successful execution for all three activities.

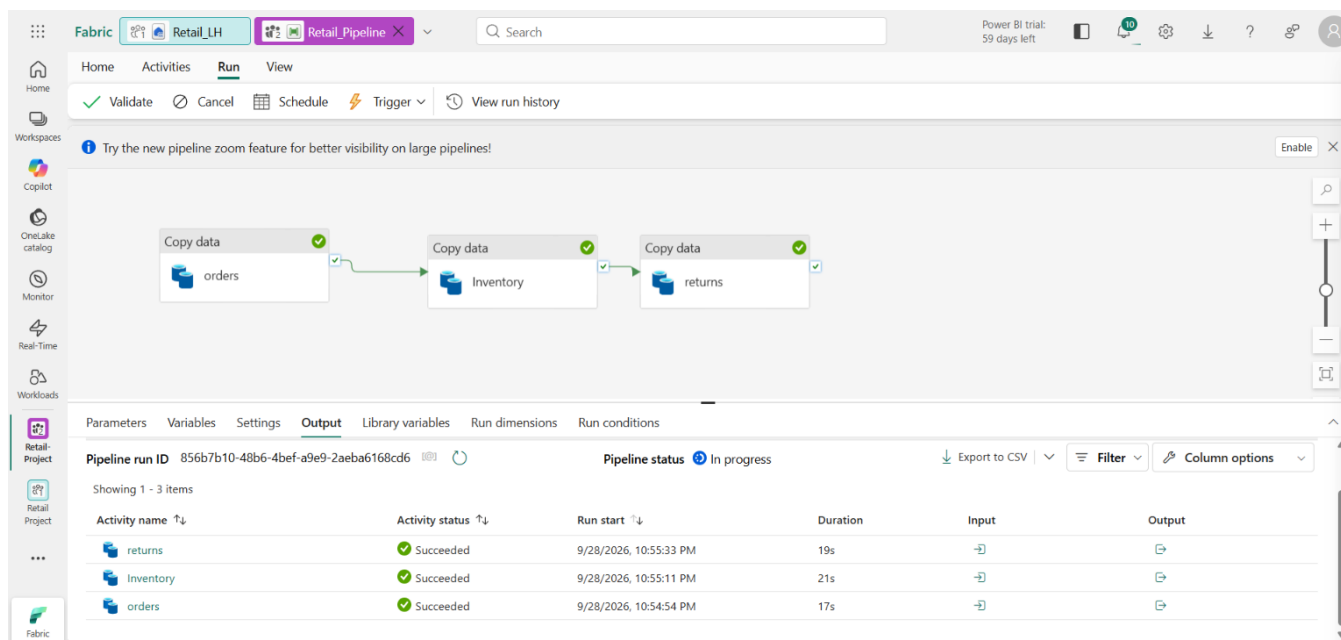


Figure 6. Pipeline design and execution result.

Project significance: This step connects the implementation stage to the next stage in the data flow, so the report can be followed from source ingestion through transformation and reporting.

7. Bronze Layer – Initial Landing

The Bronze area of Retail_LH contains Parquet representations of the incoming files. The notebook can access these files through OneLake. Bronze preserves the source information before business transformations.

The screenshot displays the Microsoft Fabric interface. On the left, the 'Explorer' pane shows the 'Retail_LH' workspace with a 'Bronze' folder under 'Files'. The 'Files' pane lists three files: 'inventory_data.parquet', 'orders_data.parquet', and 'returns_data.xlsx.parquet'. The main notebook area shows two code cells. The first cell reads Parquet data from a file in OneLake. The second cell displays the resulting DataFrame. Below the code, a 'Table view' of the data is shown, containing 7 columns and 15 rows. The status bar at the bottom indicates 'Session ready' and 'AutoSave: On'.

ABC product_id	ABC productName	ABC stock	ABC last_stocked	ABC warehouse	ABC cost
P001	Apple iPhone13	25 units	2023/06/01	Delhi_1	\$700
P002	Samsung Galaxy ...	12	01-07-2023	BLR#2	INR 720
P003	Oneplus Nord	twenty	2023.06.30	mumbai@3	Rs. 2500
P004	Nokia 3310		2023/07/05		NULL
P005	Xiaomi note 12 pro	55	07-12-2023	hydzone2	\$299

Figure 7. Bronze area and initial inventory read.

Project significance: This step connects the implementation stage to the next stage in the data flow, so the report can be followed from source ingestion through transformation and reporting.

8. Inspect Raw Orders Data

The raw Orders preview shows the original fields and several data-quality variations, including inconsistent product-name capitalization, mixed quantity representations, multiple date formats, currency symbols/codes, and null values.

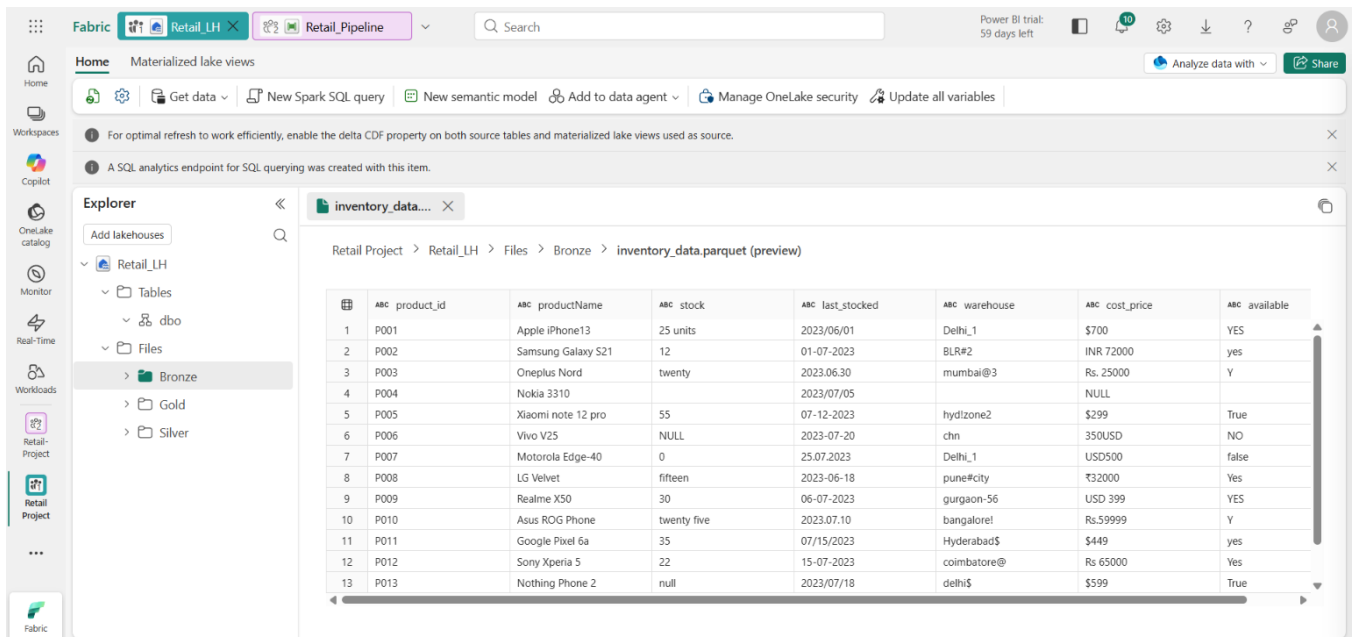
	ABC Order_ID	ABC cust_id	ABC Product_Na...	ABC Qty	ABC Order_Date	ABC Order_Amo...	ABC Delivery_Sta...	ABC Payment_M...	ABC Ship_Address	ABC
1	1001	C001	Apple iPhone13	one	2023/06/12	\$799.0	delivered	CreditCard	2218, baker st., L...	john.d
2	1002	C_002	samsung Galaxy ...	NULL	12-07-2023	\$1200	Delivered	PayPal	Noida #Sector21	NULL
3	1003	C003	NULL	3	2023.07.15	NULL	Pending	DEBIT card	123, main street	jane_c
4	1004	C004	Oneplus Nord	ONE	15/07/2023	499 USD	delivrd	upi	mumbai	peter,j
5	1005	NULL	Nokia 3310	1	07-20-2023	₹49	Delivered	Credit card	Delhi#4	bruce.
6	1006	C006	Xiaomi note 12 ...	02	07.21.2023	\$899.99	pending	NULL	NULL	NULL
7	1007	C007	Vivo V25	3	NULL	375.0	NULL	Credit_Card	chennai	clarkkk
8	1008	C008	NULL	4	2023/13/07	899.1 INR	DELIVERD	UPI	New Delhi	batma
9	1009	C009	iPhone, 14 Pro ...	1	July 20th 2023	\$1099	On The Way	CREDIT card	Gotham City	joker#
10	1010	C010	Motorola Edge-40	NaN	2023.07.25	USD 550	OnHold	NULL	N/A	brucei
11	1011	C011	LG Velvet	2	2023/06/18	₹35000	Delivered	paypal	pune#city	lguser
12	1012	C012	Realme X50	NULL	06-07-2023	USD 399	pending	Credit Card	sector-56 gurga...	realm
13	1013	C013	Asus ROG Phone	Three	2023.07.10	Rs.59999	Delivered	cash	bangalorel	rog@i
14	1014	C014	Google Pixel 6a	1	07/15/2023	\$449	Cancelled	debit	Hyderabad\$	pixel@

Figure 8. Raw orders preview.

Project significance: This step connects the implementation stage to the next stage in the data flow, so the report can be followed from source ingestion through transformation and reporting.

9. Inspect Raw Inventory Data

The raw inventory preview contains product_id, productName, stock, last_stocked, warehouse, cost_price and available. Values such as 'twenty', 'fifteen', NULL, different date formats and mixed currency formats demonstrate the need for cleaning.



	ABC product_id	ABC productName	ABC stock	ABC last_stocked	ABC warehouse	ABC cost_price	ABC available
1	P001	Apple iPhone13	25 units	2023/06/01	Delhi_1	\$700	YES
2	P002	Samsung Galaxy S21	12	01-07-2023	BLR#2	INR 72000	yes
3	P003	Oneplus Nord	twenty	2023.06.30	mumbai@3	Rs. 25000	Y
4	P004	Nokia 3310		2023/07/05		NULL	
5	P005	Xiaomi note 12 pro	55	07-12-2023	hydizone2	\$299	True
6	P006	Vivo V25	NULL	2023-07-20	chn	350USD	NO
7	P007	Motorola Edge-40	0	25.07.2023	Delhi_1	USD500	false
8	P008	LG Velvet	fifteen	2023-06-18	pune#city	₹32000	Yes
9	P009	Realme X50	30	06-07-2023	gurgaon-56	USD 399	YES
10	P010	Asus ROG Phone	twenty five	2023.07.10	bangalore1	Rs.59999	Y
11	P011	Google Pixel 6a	35	07/15/2023	Hyderabad\$	\$449	yes
12	P012	Sony Xperia 5	22	15-07-2023	coimbatore@	Rs 65000	Yes
13	P013	Nothing Phone 2	null	2023/07/18	delhi\$	\$599	True

Figure 9. Raw inventory preview.

Project significance: This step connects the implementation stage to the next stage in the data flow, so the report can be followed from source ingestion through transformation and reporting.

10. Inspect Raw Returns Data

The returns dataset contains return identifiers, order and customer references, product, return reason, return date, refund status, pickup address and return amount. The screenshot also shows an initial header-like row that requires careful handling.

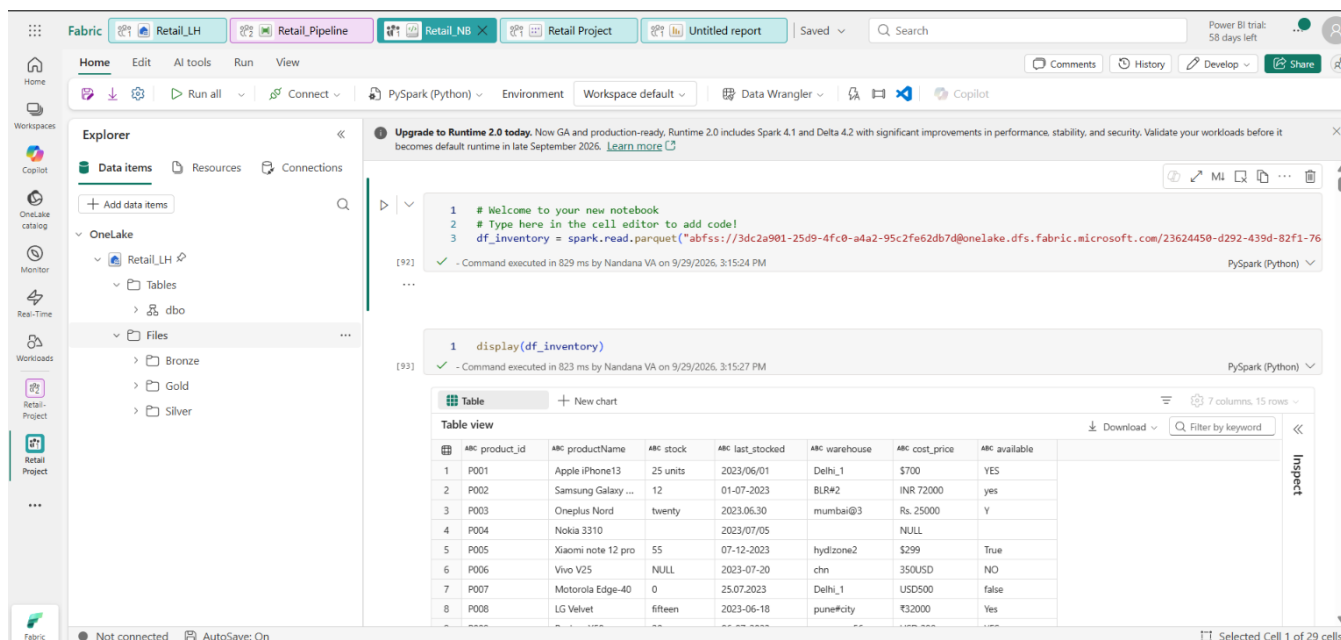
	ABC Prop_0	ABC Prop_1	ABC Prop_2	ABC Prop_3	ABC Prop_4	ABC Prop_5	ABC Prop_6	ABC Prop_7	ABC Prop_8
1	Return_ID	Order_ID	Customer_ID	Product	Return_Reason	Return_Date	Refund_Status	Pickup_Address	Return_Amount
2	R001	1001	C001	apple iphone13	Defective	15-07-2023	Done	221B Baker St Lon...	\$799
3	R002	1002	C_002	samsung GALAXY ...	CHANGED mind	2023/07/20	Processing	Sector-21 Noida	₹1200
4	R003	1004	C004	oneplus NORD	Late Delivery	2023.07.22	in progress	mumbai	499
5	R004	1005	NULL	nokia 3310	not as described	20.07.2023	Completed	delhi-4	Rs 49
6	R005	1006	C006	xiaomi note 12 pro	Wrong product	07/25/2023	NULL	hyderabad	899.99
7	R006	1007	C007	VIVO V25	Damaged Box	July 26 2023	PENDING	chennai	\$375
8	R007	1008	C008	iphone 14 pro max	FRAUD	2023/07/28	Unknown	New Delhi	1099 USD
9	R008	1010	C010	Motorola Edge-40	Misplaced	25.07.2023	NULL	N/A	550
10	R009	1011	C011	LG Velvet	battery issue	18-06-2023	requested	pune#city	₹35000
11	R010	1012	C012	Realme X50	heating issue	06.07.2023	pending	sector-56 gurgaon	USD 399
12	R011	1013	C013	Asus ROG Phone	Overheating	2023-07-10	Done	bangalorel	Rs.59999
13	R012	1014	C014	Google Pixel 6a	screen defect	15/07/2023	Issued	Hyderabad\$	\$449
14	R013	1015	C015	Sony Xperia 5	Late	15-07-2023	completed	coimbatore@	Rs 65000
15	R014	NULL	NULL	NULL	NULL	NULL	null	NULL	NULL

Figure 10. Raw returns preview.

Project significance: This step connects the implementation stage to the next stage in the data flow, so the report can be followed from source ingestion through transformation and reporting.

11. Read Bronze Data with PySpark

The Retail_NB notebook reads Bronze Parquet data using PySpark and displays the inventory dataframe. This establishes the notebook as the transformation layer between raw Bronze files and curated Silver/Gold outputs.



The screenshot shows a Microsoft Fabric notebook interface. The left sidebar contains the Explorer pane with a tree view showing the workspace structure: OneLake > Retail_LH > Tables > dbo > Files > Bronze > Gold > Silver. The main area displays two code cells. The first cell contains PySpark code to read a Parquet file from OneLake. The second cell displays the resulting dataframe. Below the code, a 'Table view' is shown with 7 columns and 15 rows. The table contains product information, stock levels, warehouse locations, and costs.

```
1 # Welcome to your new notebook
2 # Type here in the cell editor to add code!
3 df_inventory = spark.read.parquet("abfss://3dc2a901-25d9-4fc0-a4a2-95c2fe62db7d@onelake.dfs.fabric.microsoft.com/23624450-d292-439d-82f1-76")
```

```
1 display(df_inventory)
```

ABC product_id	ABC productName	ABC stock	ABC last_stocked	ABC warehouse	ABC cost_price	ABC available
P001	Apple iPhone13	25 units	2023/06/01	Delhi_1	\$700	YES
P002	Samsung Galaxy ...	12	01-07-2023	BLR#2	INR 72000	yes
P003	Oneplus Nord	twenty	2023.06.30	mumbai@3	Rs. 25000	Y
P004	Nokia 3310		2023/07/05		NULL	
P005	Xiaomi note 12 pro	55	07-12-2023	hydizone2	\$299	True
P006	Vivo V25	NULL	2023-07-20	chn	350USD	NO
P007	Motorola Edge-40	0	25.07.2023	Delhi_1	USD500	false
P008	LG Velvet	fifteen	2023-06-18	pune@city	₹32000	Yes

Figure 11. PySpark notebook reading Bronze inventory.

Project significance: This step connects the implementation stage to the next stage in the data flow, so the report can be followed from source ingestion through transformation and reporting.

12. Clean the Inventory Dataset

The cleaned inventory output standardizes the visible fields into product_id, ProductName, Stock, LastStocked, Warehouse, CostPrice and Available. The screenshot shows typed/normalized values while retaining unresolved values where appropriate.

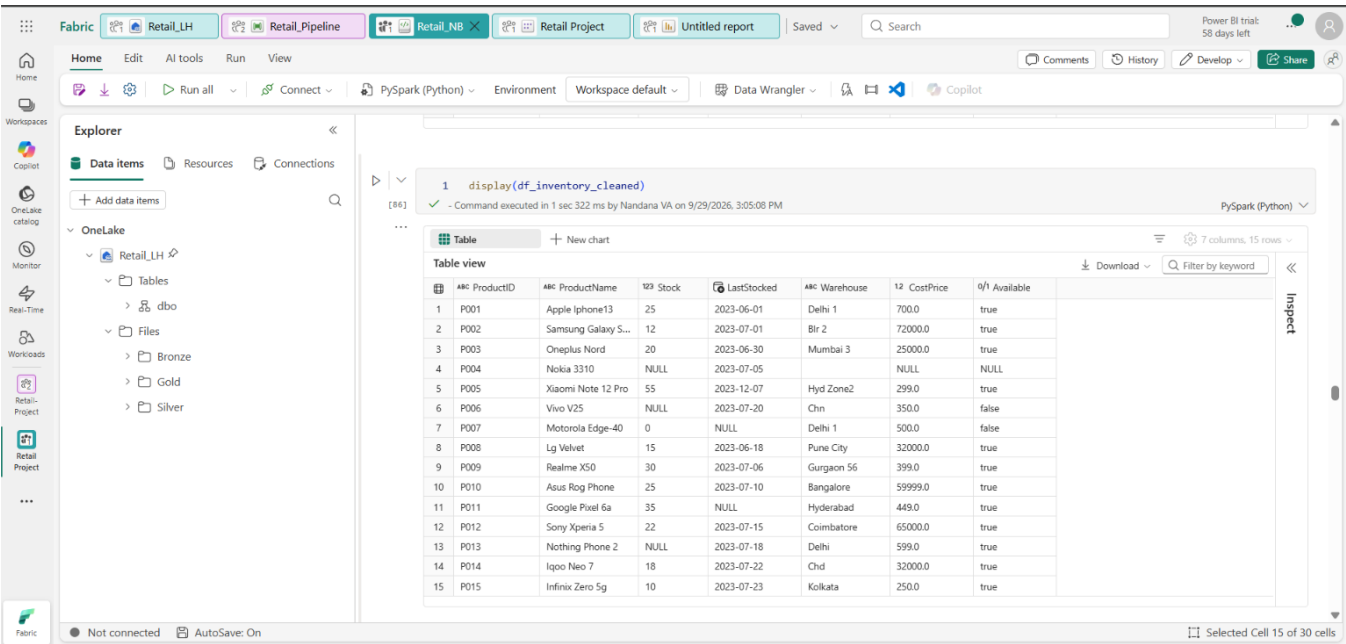


Figure 12. Cleaned inventory output.

Project significance: This step connects the implementation stage to the next stage in the data flow, so the report can be followed from source ingestion through transformation and reporting.

13. Clean the Orders Dataset

The notebook produces a cleaned orders dataset with standardized fields such as Order_ID, CustomerID, ProductName, Qty, Order_Date, OrderAmount, Delivery_Status, Payment_Mode and Ship_Address. The visible output contains 13 rows.

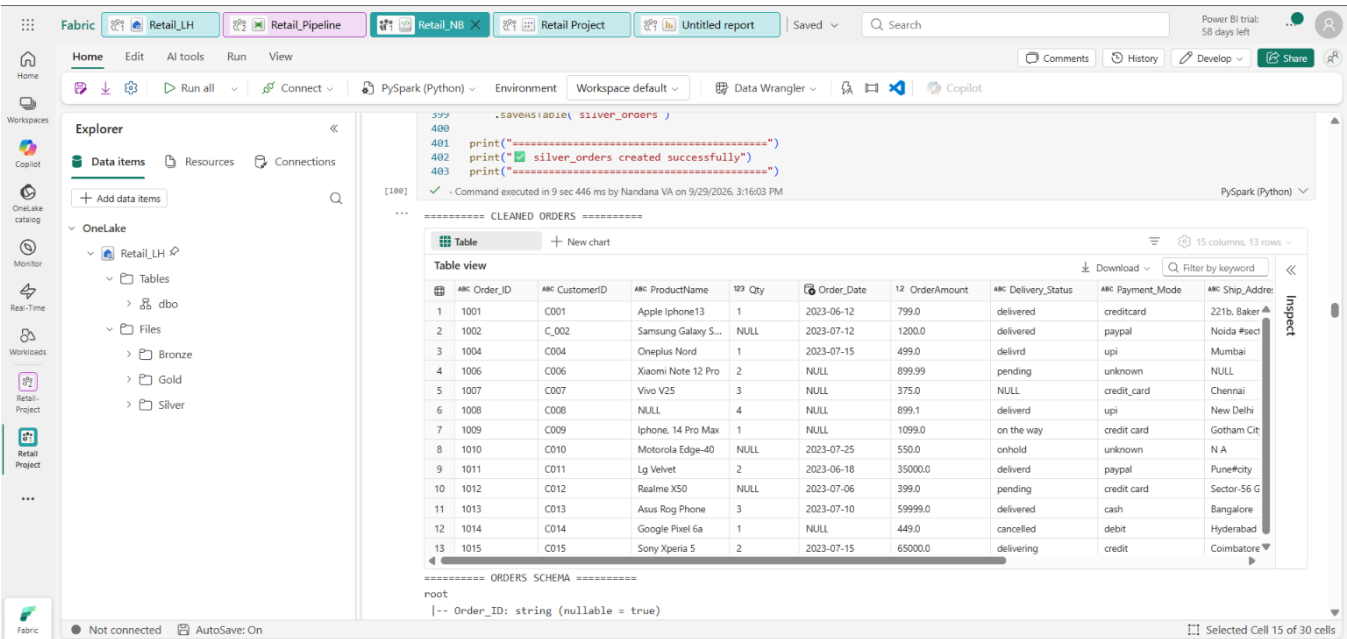
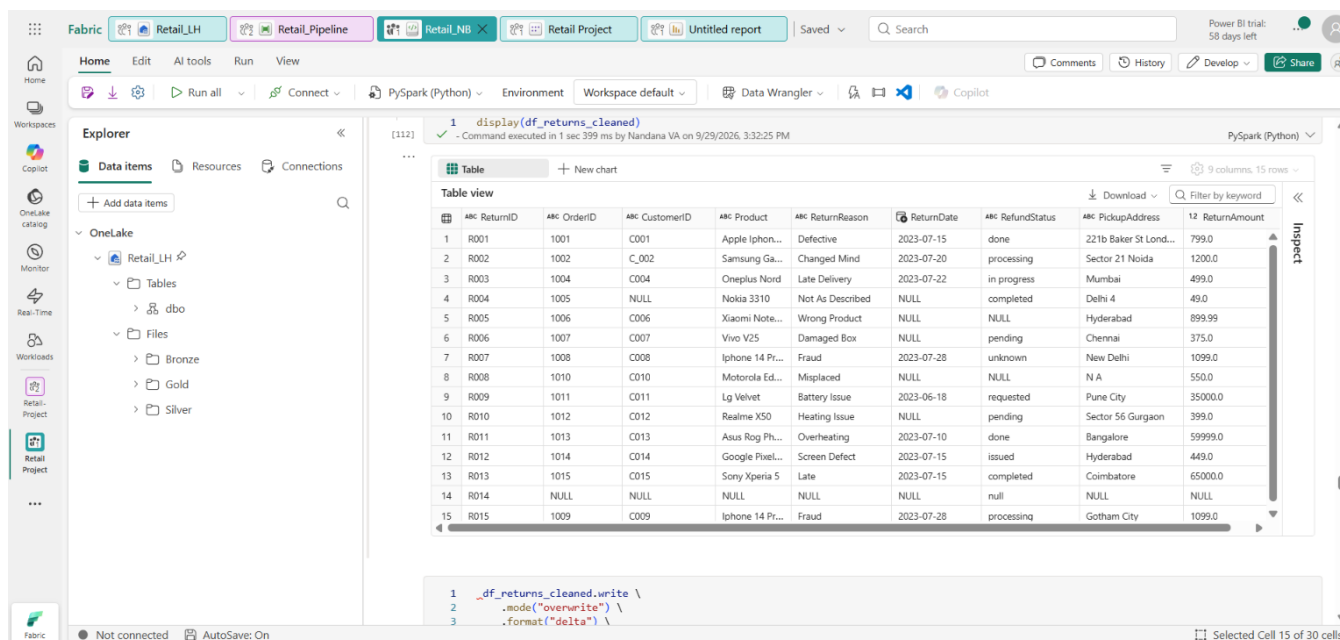


Figure 13. Cleaned orders output.

Project significance: This step connects the implementation stage to the next stage in the data flow, so the report can be followed from source ingestion through transformation and reporting.

14. Clean the Returns Dataset

The cleaned returns dataframe contains ReturnID, OrderID, CustomerID, Product, ReturnReason, ReturnDate, RefundStatus, PickupAddress and ReturnAmount. The screenshot shows 15 rows in this intermediate dataframe.



The screenshot displays the Microsoft Fabric workspace interface. On the left, the 'Explorer' pane shows the 'Retail_LH' workspace with a folder structure including 'Tables', 'dbo', 'Files', 'Bronze', 'Gold', and 'Silver'. The main area shows a PySpark notebook with a table view of the 'df_returns_cleaned' dataframe. The table contains 15 rows of data, including ReturnID, OrderID, CustomerID, Product, ReturnReason, ReturnDate, RefundStatus, PickupAddress, and ReturnAmount. The bottom of the notebook shows a code cell with the following PySpark code:

```
1 df_returns_cleaned.write \
2   .mode("overwrite") \
3   .format("delta") \
```

	ReturnID	OrderID	CustomerID	Product	ReturnReason	ReturnDate	RefundStatus	PickupAddress	ReturnAmount
1	R001	1001	C001	Apple Iphon...	Defective	2023-07-15	done	221b Baker St Lond...	799.0
2	R002	1002	C_002	Samsung Ga...	Changed Mind	2023-07-20	processing	Sector 21 Noida	1200.0
3	R003	1004	C004	Oneplus Nord	Late Delivery	2023-07-22	in progress	Mumbai	499.0
4	R004	1005	NULL	Nokia 3310	Not As Described	NULL	completed	Delhi 4	49.0
5	R005	1006	C006	Xiaomi Note...	Wrong Product	NULL	NULL	Hyderabad	899.99
6	R006	1007	C007	Vivo V25	Damaged Box	NULL	pending	Chennai	375.0
7	R007	1008	C008	Iphone 14 Pr...	Fraud	2023-07-28	unknown	New Delhi	1099.0
8	R008	1010	C010	Motorola Ed...	Misplaced	NULL	NULL	N A	550.0
9	R009	1011	C011	Lg Velvet	Battery Issue	2023-06-18	requested	Pune City	35000.0
10	R010	1012	C012	Realme X50	Heating Issue	NULL	pending	Sector 56 Gurgaon	399.0
11	R011	1013	C013	Asus Rog Ph...	Overheating	2023-07-10	done	Bangalore	59999.0
12	R012	1014	C014	Google Pixel...	Screen Defect	2023-07-15	issued	Hyderabad	449.0
13	R013	1015	C015	Sony Xperia 5	Late	2023-07-15	completed	Coimbatore	65000.0
14	R014	NULL	NULL	NULL	NULL	NULL	null	NULL	NULL
15	R015	1009	C009	Iphone 14 Pr...	Fraud	2023-07-28	processing	Gotham City	1099.0

Figure 14. Cleaned returns output.

Project significance: This step connects the implementation stage to the next stage in the data flow, so the report can be followed from source ingestion through transformation and reporting.

15. Apply Orders Cleaning Logic

The notebook code demonstrates explicit transformation logic for the Orders dataset, including renaming fields to standard business names. This stage is where inconsistent source naming is converted into a consistent analytical schema.

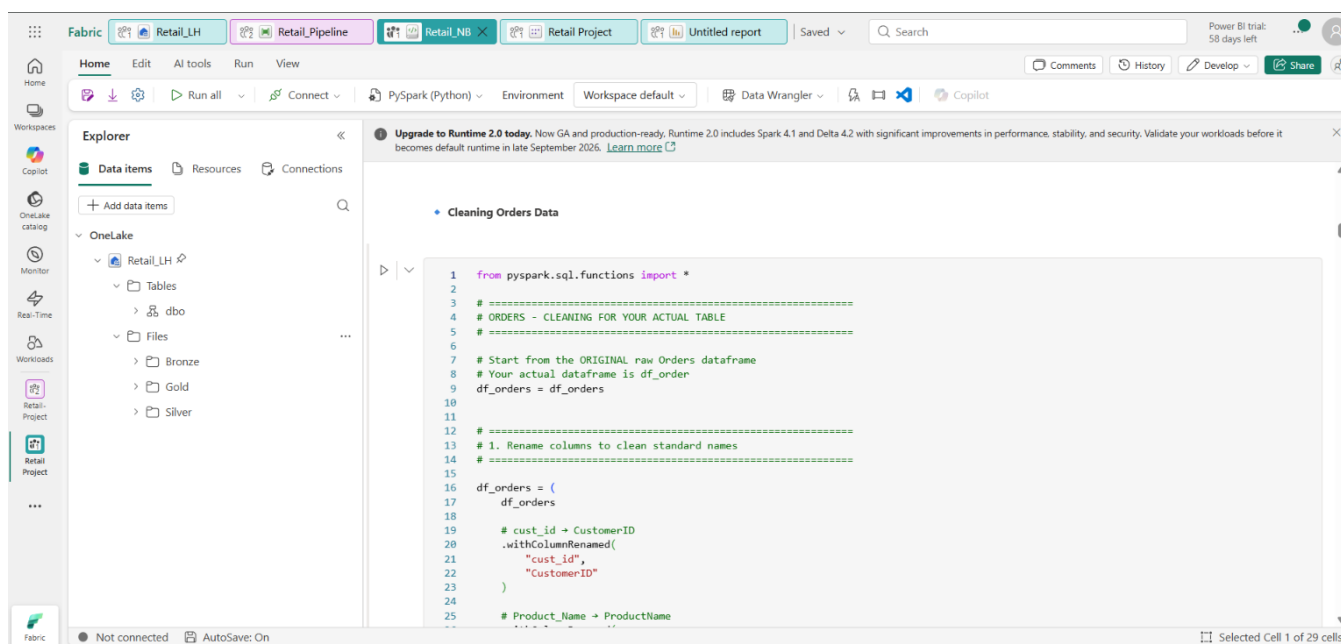
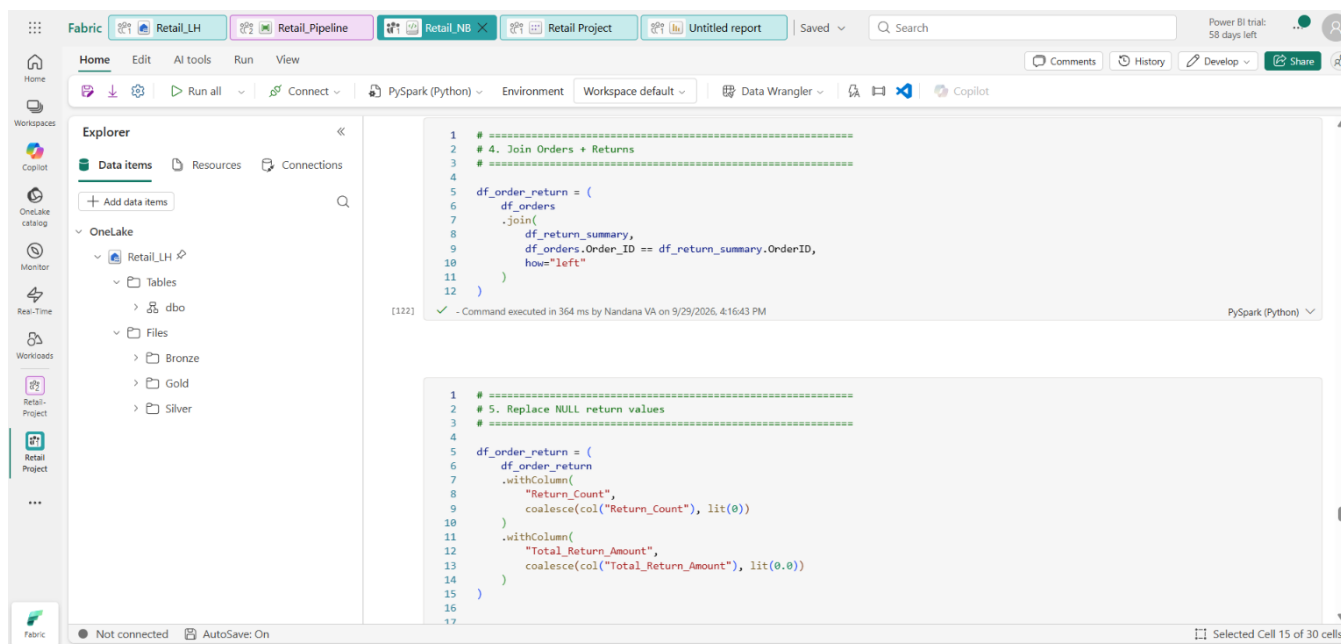


Figure 15. Orders-cleaning transformation code.

Project significance: This step connects the implementation stage to the next stage in the data flow, so the report can be followed from source ingestion through transformation and reporting.

16. Join Orders and Returns and Handle Nulls

The notebook joins the Orders dataframe with a return summary using Order_ID and applies null handling to return count and return amount fields. This produces an enriched order-level dataset suitable for KPI calculations.



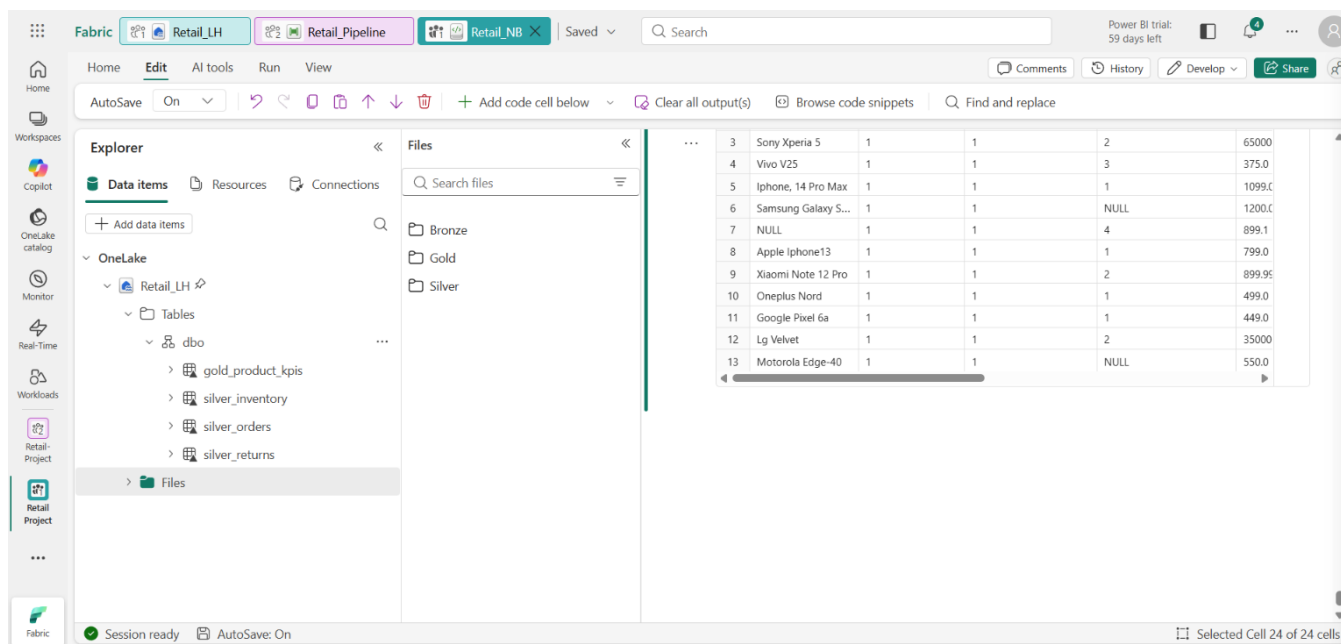
```
1 # =====  
2 # 4. Join Orders + Returns  
3 # =====  
4  
5 df_order_return = (  
6     df_orders  
7     .join(  
8         df_return_summary,  
9         df_orders.Order_ID == df_return_summary.OrderID,  
10        how="left"  
11    )  
12 )  
[122] ✓ - Command executed in 364 ms by Nandana VA on 9/29/2026, 4:16:43 PM  
PySpark (Python) ✓  
  
1 # =====  
2 # 5. Replace NULL return values  
3 # =====  
4  
5 df_order_return = (  
6     df_order_return  
7     .withColumn(  
8         "Return_Count",  
9         coalesce(col("Return_Count"), lit(0))  
10    )  
11    .withColumn(  
12        "Total_Return_Amount",  
13        coalesce(col("Total_Return_Amount"), lit(0.0))  
14    )  
15 )  
16  
17
```

Figure 16. Orders/returns join and null replacement.

Project significance: This step connects the implementation stage to the next stage in the data flow, so the report can be followed from source ingestion through transformation and reporting.

17. Create and Organize Silver and Gold Tables

The Lakehouse explorer shows silver_orders, silver_returns and silver_inventory together with the Gold table gold_product_kpis. This demonstrates the project's Bronze–Silver–Gold organization inside the Retail_LH Lakehouse.



	Product	Quantity	Price	Total Value
3	Sony Xperia 5	1	1	65000
4	Vivo V25	1	1	375.0
5	Iphone, 14 Pro Max	1	1	1099.0
6	Samsung Galaxy S...	1	1	1200.0
7	NULL	1	1	899.1
8	Apple Iphone13	1	1	799.0
9	Xiaomi Note 12 Pro	1	1	899.95
10	Oneplus Nord	1	1	499.0
11	Google Pixel 6a	1	1	449.0
12	Lg Velvet	1	1	35000
13	Motorola Edge-40	1	1	550.0

Figure 17. Silver and Gold objects in Retail_LH.

Project significance: This step connects the implementation stage to the next stage in the data flow, so the report can be followed from source ingestion through transformation and reporting.

18. Build the Power BI Semantic Model

The semantic model exposes gold_product_kpis and its analytical fields, including Avg_Cost, Avg_Order_Value, Current_Stock, Net_Profit, Return_Rate_Percent, Returned_Orders, Total_COGS, Total_Orders, Total_Quantity, Total_Return_Amount, Total>Returns, Total_Revenue and Unique_Customers.

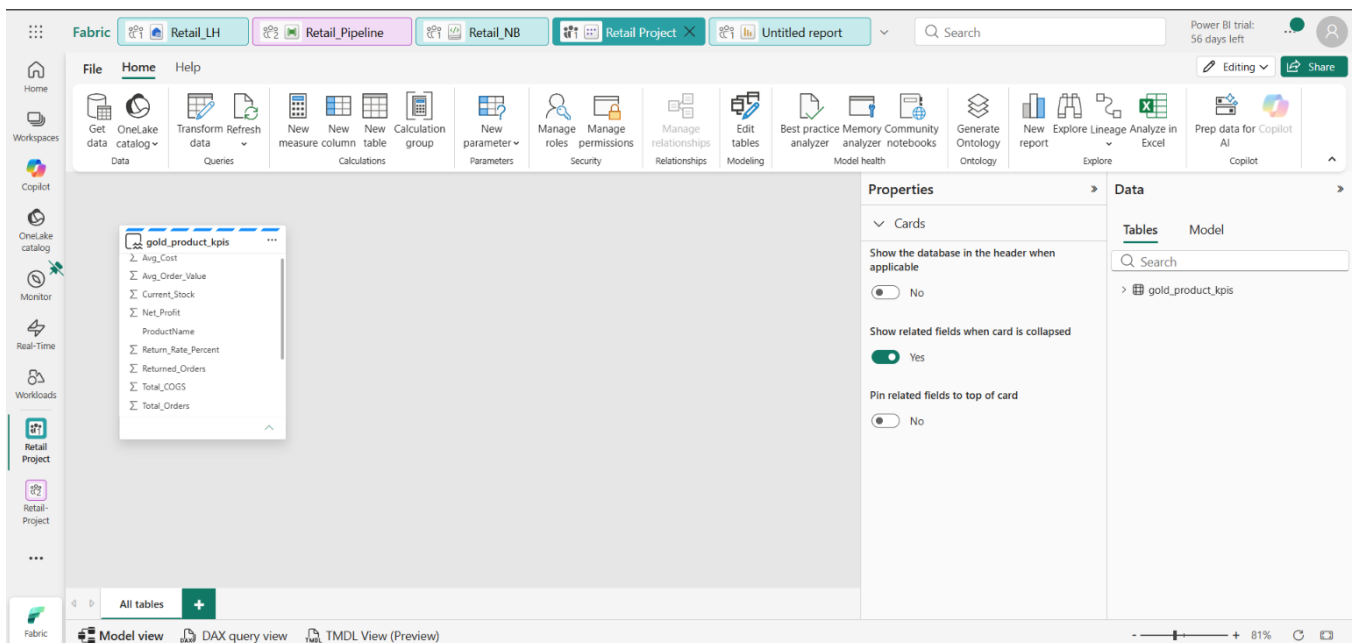


Figure 18. Power BI model view with gold_product_kpis.

Project significance: This step connects the implementation stage to the next stage in the data flow, so the report can be followed from source ingestion through transformation and reporting.

19. Develop the Power BI Report

The report page is assembled from the Gold/semantic-model fields. The visible design includes a ProductName slicer, stock analysis, returned-orders analysis, product-level summary data and average-cost analysis.

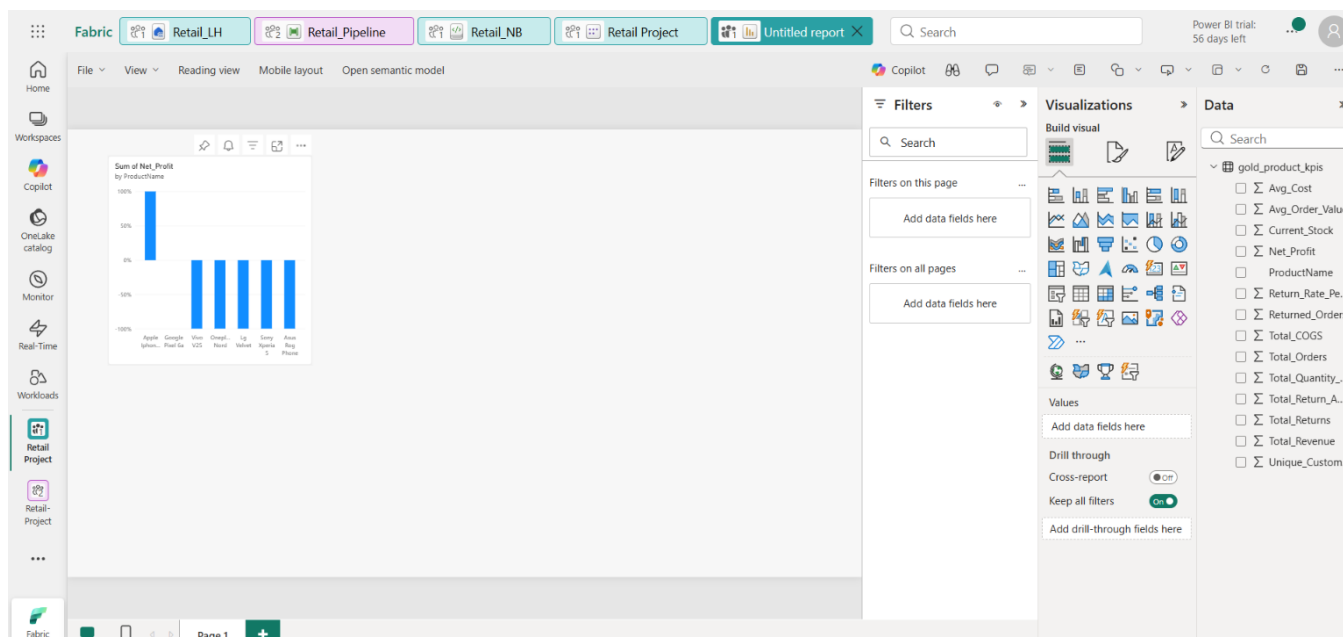


Figure 19. Power BI report development view.

Project significance: This step connects the implementation stage to the next stage in the data flow, so the report can be followed from source ingestion through transformation and reporting.

20. Final Retail Sales Performance Dashboard

The completed report is presented as the Retail Sales Performance Dashboard. The final page contains a ProductName filter, Current Stock by ProductName, Returned Orders by ProductName, a product-level table with order, quantity, return amount, returns and average-cost fields, and an average-cost visual.

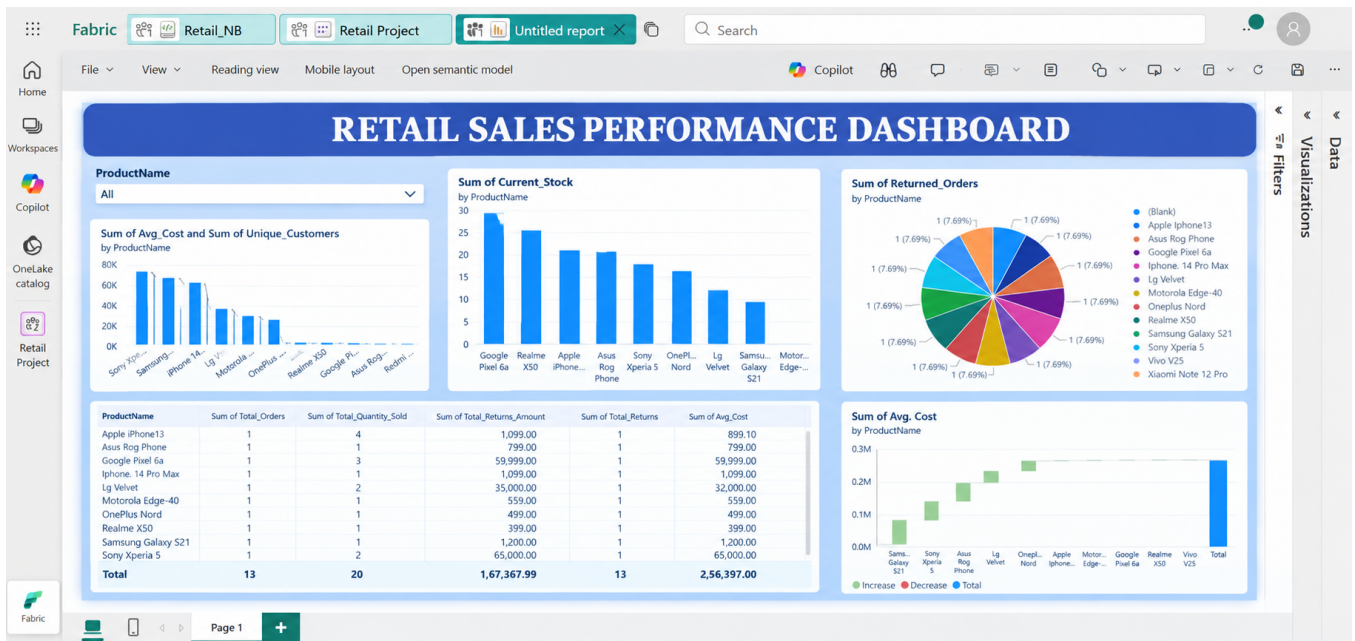


Figure 20. Final Power BI dashboard.

Project significance: This step connects the implementation stage to the next stage in the data flow, so the report can be followed from source ingestion through transformation and reporting.

21. Data Quality and Validation Notes

The screenshots clearly show several source-data inconsistencies. These are important because a dashboard can look technically correct while still containing ambiguous business values. The following controls should be maintained when the project is presented or extended.

Issue observed	Treatment / reporting note
Mixed quantity values	Normalize approved numeric words and numeric strings; retain unresolved values for review.
Mixed date formats	Parse supported formats explicitly and flag invalid dates.
Mixed currency representations	Keep currency context separate; do not treat symbol removal as currency conversion.
Inconsistent capitalization/naming	Standardize known product and field names during Silver transformation.
Null / missing values	Use controlled null handling; avoid silently turning unknown values into zero unless justified.
15 raw vs. 13 cleaned Orders rows	Reconcile accepted/rejected/duplicate records before production use.
Return preview validation	Verify the dataframe and source path whenever a displayed schema does not match the expected data

22. Final Dashboard Components

- ProductName slicer for interactive filtering.
- Current Stock by ProductName column chart.
- Returned Orders by ProductName pie chart.
- Product-level summary table containing orders, quantity sold, return amount, returns and average-cost fields.
- Average-cost visual by ProductName.
- Microsoft Fabric/Power BI report interface with the final heading: RETAIL SALES PERFORMANCE DASHBOARD.

23. Conclusion

The completed project demonstrates a practical Microsoft Fabric analytics pipeline from raw retail files to an interactive Power BI report. The strongest implementation evidence is the source container, successful pipeline run, Bronze data, PySpark cleaning outputs, Silver/Gold Lakehouse objects, semantic model and final dashboard. Before treating the dashboard as production-grade business reporting, the remaining data-quality and metric-validation items should be reconciled, especially row-count differences, currency handling, return-rate definitions and any aggregation that can duplicate orders.

Public GitHub safety note

The workspace-creation screenshot and Azure access-key screenshot are not reproduced in this public report because they contain account/credential-related information. The implementation steps they represent are documented in text without exposing those details.